МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №7

по дисциплине «Вычислительная математика»

Тема: Решение систем линейных уравнений методом Гаусса.

Студент гр. 4342 Тиссен М.Ю.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Реализовать программу для решения систем линейных алгебраических уравнений методом Гаусса с частичным выбором ведущего элемента по столбцу, исследовать ее работу на матрицах разных типов и размеров, а также оценить точность получаемого решения по сравнению с заранее известным точным решением и по величине невязки.

Теоретический минимум.

Метод Гаусса - это один из основных прямых методов решения систем линейных уравнений. Его идея состоит в том, чтобы с помощью элементарных преобразований строк последовательно исключить неизвестные из уравнений и привести матрицу системы к верхнетреугольному виду. После этого значения неизвестных находятся обратной подстановкой.

В данной работе используется модификация метода Гаусса с частичным выбором ведущего элемента по столбцу. На каждом шаге среди элементов текущего столбца, начиная с текущей строки, выбирается элемент с наибольшим модулем. Строка, содержащая этот элемент, меняется местами с текущей строкой. Такой подход уменьшает влияние ошибок округления и делает вычисления более устойчивыми.

Если на некотором шаге в текущем столбце не удается выбрать ненулевой ведущий элемент, это означает, что ранг матрицы меньше числа неизвестных. В таком случае система может либо иметь бесконечно много решений, либо не иметь решений вовсе. Если после исключения появляется строка, в которой все коэффициенты равны нулю, а соответствующий элемент правой части не равен нулю, система несовместна. Если противоречия нет, система имеет бесконечно много решений. Если же ведущие элементы находятся на всех шагах, система имеет единственное решение. Именно такие статусы и предусмотрены в реализации программы.

Для плотной матрицы размера n x n вычислительная сложность метода Гаусса составляет порядка O(n^3), более точно - около $\frac{2}{3}n^3$ арифметических операций. Затраты памяти составляют O(n^2), так как в программе используется хранение плотной матрицы.

Выполнение работы.

В работе метод проверяется на системах разных типов: на корректных системах с единственным решением, на примерах, где требуется перестановка строк, а также на вырожденных совместных и несовместных системах. Кроме того, рассматриваются матрицы Гильберта и большие диагонально доминирующие матрицы, что позволяет оценить поведение метода как на плохо обусловленных, так и на более устойчивых системах.

Для тех случаев, где точное решение известно заранее, формируется правая часть системы, после чего найденный вектор сравнивается с точным. Дополнительно вычисляется невязка, то есть разность между левой и правой частями системы после подстановки найденного решения. Это позволяет проверить не только сам факт получения ответа, но и его точность.

Оценка ошибки решения и невязки выполняется в трех нормах: L1, L2, Linf. Норма L1 показывает суммарную абсолютную величину отклонений по всем компонентам, поэтому она удобна для общей оценки накопленной ошибки. Норма L2 характеризует среднюю величину отклонения в евклидовом смысле и позволяет судить о точности результата в целом. Норма Linf показывает максимальную ошибку среди всех компонент, то есть отражает наихудший случай. Использование сразу трех норм дает более полное представление о качестве полученного решения. Также в ходе экспериментов фиксируется время работы метода, что позволяет оценить его поведение при увеличении размера матрицы.

Выводы.

В ходе работы был исследован метод Гаусса с частичным выбором ведущего элемента по столбцу для решения систем линейных алгебраических уравнений. Проверка на различных тестах показала, что данный метод позволяет успешно находить решение для систем с единственным решением, а также определять случаи бесконечного числа решений или отсутствия решений. Использование частичного выбора ведущего элемента повышает устойчивость вычислений и делает метод более надежным при работе с разными типами матриц.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <clocale>
#include <algorithm>
#include <cstdint>
#include <cmath>
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <limits>
#include <random>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

#include "methods.h"

#ifdef _WIN32
#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>
#endif

namespace {

constexpr double kZeroTolerance = 1e-12;
constexpr std::size_t kVeryLargeN = 10000;
constexpr std::size_t kFullMatrixLimit = 10;
constexpr std::size_t kPreviewRows = 6;
constexpr std::size_t kPreviewCols = 6;
constexpr std::size_t kPreviewVectorItems = 8;
constexpr std::size_t kConditionNumberLimit = 30;

struct ExperimentReport {
    std::string name;
    std::string matrixType;
    std::string note;
    std::string systemPreview;
    std::uint64_t seed = 0;
    std::size_t n = 0;
    bool executed = false;
    SolutionStatus status = SolutionStatus::NoSolution;
    PivotStrategy pivotStrategy = PivotStrategy::PartialByColumn;
    std::size_t rank = 0;
    std::size_t rowSwaps = 0;
    double elapsedMs = 0.0;
    double conditionNumberInf = std::numeric_limits<double>::quiet_NaN();
    std::string conditionNote;
    VectorNorms solutionError{
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN()
    };
    VectorNorms residual{
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN(),
```

```
        std::numeric_limits<double>::quiet_NaN()
    };
};

void SetupConsole() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif
    std::setlocale(LC_ALL, ".UTF-8");
}

bool HasFlag(int argc, char* argv[], const std::string& flag) {
    for (int index = 1; index < argc; ++index) {
        if (flag == argv[index]) {
            return true;
        }
    }
    return false;
}

std::uint64_t GetRunSeed(int argc, char* argv[]) {
    const std::string prefix = "--seed=";
    for (int index = 1; index < argc; ++index) {
        const std::string argument = argv[index];
        if (argument.rfind(prefix, 0) == 0) {
            return std::stoull(argument.substr(prefix.size()));
        }
        if (argument == "--seed" && index + 1 < argc) {
            return std::stoull(argv[index + 1]);
        }
    }

    std::random_device device;
    const std::uint64_t high =
        static_cast<std::uint64_t>(device()) << 32;
    const std::uint64_t low = static_cast<std::uint64_t>(device());
    return high ^ low;
}

std::uint64_t DeriveSeed(std::uint64_t seed, std::uint64_t salt) {
    return seed + 0x9E3779B97F4A7C15ULL * salt;
}

DenseMatrix MakeRandomPivotRequiredMatrix(std::uint64_t seed) {
    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> distribution(-5.0, 5.0);

    for (int attempt = 0; attempt < 2000; ++attempt) {
        DenseMatrix matrix = MakeRandomDenseMatrix(3, rng());
        matrix(0, 0) = 0.0;

        if (std::abs(matrix(1, 0)) < 0.5) {
            matrix(1, 0) = (distribution(rng) >= 0.0) ? 1.0 : -1.0;
        }
        if (std::abs(matrix(2, 0)) < 0.5) {
            matrix(2, 0) = (distribution(rng) >= 0.0) ? 2.0 : -2.0;
        }

        const std::vector<double> xExact = MakeRandomVector(
            3,
            DeriveSeed(seed, static_cast<std::uint64_t>(attempt) + 1)
        );
```

```
        const GaussianResult probe = SolveGaussian(
            matrix,
            Multiply(matrix, xExact),
            kZeroTolerance
        );
        if (probe.status == SolutionStatus::Unique) {
            return matrix;
        }
    }

    throw std::runtime_error(
         "Не удалось сгенерировать случайную матрицу с обязательной перестановкой
строк."
    );
}

DenseMatrix MakeRandomRankDeficientMatrix(std::size_t n, std::uint64_t seed) {
    const std::vector<double> baseRow = MakeRandomVector(n, DeriveSeed(seed, 1));
    std::mt19937_64 rng(DeriveSeed(seed, 2));
    std::uniform_real_distribution<double> scaleDistribution(-3.0, 3.0);

    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
        double scale = scaleDistribution(rng);
        if (std::abs(scale) < 0.5) {
            scale = (scale < 0.0) ? -1.0 : 1.0;
        }
        for (std::size_t col = 0; col < n; ++col) {
            matrix(row, col) = scale * baseRow[col];
        }
    }
    return matrix;
}

std::string FormatMatrix(const DenseMatrix& matrix) {
    const bool fullMatrix = matrix.rows() <= kFullMatrixLimit &&
                            matrix.cols() <= kFullMatrixLimit;
    const std::size_t shownRows = fullMatrix
        ? matrix.rows()
        : std::min(matrix.rows(), kPreviewRows);
    const std::size_t shownCols = fullMatrix
        ? matrix.cols()
        : std::min(matrix.cols(), kPreviewCols);

    std::ostringstream out;
    out << std::fixed << std::setprecision(6);

    if (fullMatrix) {
        out << "Матрица A:\n";
    } else {
        out << "Матрица A, показан левый верхний фрагмент "
            << shownRows << "x" << shownCols
            << " из " << matrix.rows() << "x" << matrix.cols() << ":\n";
    }

    for (std::size_t row = 0; row < shownRows; ++row) {
        out << "  [";
        for (std::size_t col = 0; col < shownCols; ++col) {
            out << std::setw(12) << matrix(row, col);
            if (col + 1 < shownCols) {
                out << ' ';
            }
        }
```

```
        if (!fullMatrix && shownCols < matrix.cols()) {
            out << " ...";
        }
        out << " ]\n";
    }

    if (!fullMatrix && shownRows < matrix.rows()) {
        out << "  ...\n";
    }

    return out.str();
}

std::string FormatVector(const std::vector<double>& values,
                         const std::string& title) {
    const bool fullVector = values.size() <= kFullMatrixLimit;
    std::ostringstream out;
    out << std::fixed << std::setprecision(6);

    if (fullVector) {
        out << title << " = [";
        for (std::size_t index = 0; index < values.size(); ++index) {
            out << values[index];
            if (index + 1 < values.size()) {
                out << ", ";
            }
        }
        out << "]\n";
        return out.str();
    }

    const std::size_t shown = std::min(values.size(), kPreviewVectorItems);
    out << title << ", первые " << shown
        << " элементов из " << values.size() << ": [";
    for (std::size_t index = 0; index < shown; ++index) {
        out << values[index];
        if (index + 1 < shown) {
            out << ", ";
        }
    }
    out << "]\n";
    return out.str();
}

std::string BuildSystemPreview(const DenseMatrix& matrix,
                               const std::vector<double>& b,
                               const std::vector<double>* exactX) {
    std::ostringstream out;
    out << FormatMatrix(matrix);
    out << FormatVector(b, "Вектор b");
    if (exactX != nullptr) {
        out << FormatVector(*exactX, "Точное решение x*");
    }
    return out.str();
}

ExperimentReport RunExperiment(const std::string& name,
                               const std::string& matrixType,
                               DenseMatrix matrix,
                               std::vector<double> b,
                               const std::vector<double>* exactX,
                               const std::string& note,
                               std::uint64_t seed) {
```

```
    const DenseMatrix originalMatrix = matrix;
    const std::vector<double> originalB = b;

    const auto start = std::chrono::steady_clock::now();
    const GaussianResult result =
        SolveGaussian(std::move(matrix), std::move(b), kZeroTolerance);
    const auto finish = std::chrono::steady_clock::now();

    ExperimentReport report;
    report.name = name;
    report.matrixType = matrixType;
    report.note = note;
    report.systemPreview = BuildSystemPreview(originalMatrix, originalB, exactX);
    report.seed = seed;
    report.n = originalMatrix.rows();
    report.executed = true;
    report.status = result.status;
    report.pivotStrategy = result.pivotStrategy;
    report.rank = result.rank;
    report.rowSwaps = result.rowSwaps;
    report.elapsedMs =
        std::chrono::duration<double, std::milli>(finish - start).count();

    if (!result.x.empty()) {
        report.residual =
            ComputeNorms(Subtract(Multiply(originalMatrix, result.x), originalB));
    }

    if (exactX != nullptr && result.status == SolutionStatus::Unique) {
        report.solutionError = ComputeNorms(Subtract(result.x, *exactX));
    }

        if (result.status == SolutionStatus::Unique) {
        if (report.n <= kConditionNumberLimit) {
            double conditionNumber = 0.0;
            if (ComputeConditionNumberInf(
                    originalMatrix,
                    kZeroTolerance,
                    conditionNumber)) {
                report.conditionNumberInf = conditionNumber;
            } else {
                report.conditionNote =
                    "не удалось вычислить cond_inf(A)";
            }
        } else {
            report.conditionNote =
                "не вычислялось: для больших матриц расчет слишком дорогой";
        }
    } else {
        report.conditionNote =
            "не вычисляется для вырожденной матрицы";
    }

    return report;
}

ExperimentReport RunExperimentWithExactSolution(const std::string& name,
                                                const std::string& matrixType,
                                                DenseMatrix matrix,
                                                const std::vector<double>& exactX,
                                                const std::string& note,
                                                std::uint64_t seed) {
    const std::vector<double> b = Multiply(matrix, exactX);
```

```
     return RunExperiment(name, matrixType, std::move(matrix), b, &exactX, note,
seed);
}

ExperimentReport MakeSkippedLargeReport(std::uint64_t seed) {
    ExperimentReport report;
    report.name = "Случайная диагонально доминирующая матрица 10000x10000";
    report.matrixType = "случайная диагонально доминирующая";
    report.note =
          "По умолчанию тест пропущен. Запустите программу с флагом --run-10000,
чтобы принудительно выполнить этот эксперимент.";
    report.seed = seed;
    report.n = kVeryLargeN;
    report.executed = false;
    return report;
}

void PrintNormLine(const std::string& label, const VectorNorms& norms) {
    std::cout << "  " << label
              << ": L1 = " << std::scientific << norms.l1
              << ", L2 = " << norms.l2
              << ", Linf = " << norms.linf << '\n';
}

void PrintReport(const ExperimentReport& report) {
    std::cout << "Случай: " << report.name << '\n';
    std::cout << "  Размер n = " << report.n
              << ", тип матрицы = " << report.matrixType << '\n';
    if (report.seed != 0) {
        std::cout << "  Seed = " << report.seed << '\n';
    }
    if (!report.systemPreview.empty()) {
        std::cout << report.systemPreview;
    }

    if (!report.executed) {
        std::cout << "  Статус = пропущен\n";
        std::cout << "  Оценка памяти только под A, МиБ = "
                  << std::fixed << std::setprecision(3)
                  << EstimateDenseMatrixMiB(report.n) << '\n';
        std::cout << "  Оценка памяти под [A|b], МиБ = "
                  << EstimateAugmentedMatrixMiB(report.n) << '\n';
        std::cout << "  Оценка числа операций Гаусса = "
                  << std::scientific << EstimateGaussianFlops(report.n) << '\n';
        std::cout << "  Примечание = " << report.note << "\n\n";
        return;
    }

    std::cout << "  Статус = " << ToString(report.status)
              << ", ранг = " << report.rank << "/" << report.n
              << ", перестановок строк = " << report.rowSwaps
              << ", время, мс = " << std::fixed << std::setprecision(3)
              << report.elapsedMs << '\n';
    std::cout << "  Стратегия выбора ведущего элемента = "
              << ToString(report.pivotStrategy) << '\n';
    PrintNormLine("невязка", report.residual);
        if (!std::isnan(report.conditionNumberInf)) {
        std::cout << "  cond_inf(A) = "
                  << std::scientific << report.conditionNumberInf << '\n';
    } else if (!report.conditionNote.empty()) {
        std::cout << "  cond_inf(A): " << report.conditionNote << '\n';
    }
```

```
    if (!std::isnan(report.solutionError.l2)) {
        PrintNormLine("ошибка решения", report.solutionError);
    }

    if (!report.note.empty()) {
        std::cout << "  Примечание = " << report.note << '\n';
    }
    std::cout << '\n';
}

void WriteCsv(const std::string& filename,
              const std::vector<ExperimentReport>& reports) {
    std::ofstream out(filename);
                                                          out                <<
"случай;seed;тип_матрицы;n;выполнен;статус;ранг;перестановки_строк;время_мс;"
       "cond_inf;комментарий_cond;ошибка_l1;ошибка_l2;ошибка_linf;"
       "невязка_l1;невязка_l2;невязка_linf;примечание\n";
    out << std::setprecision(17);

    for (const auto& report : reports) {
        out << report.name << ';'
            << report.seed << ';'
            << report.matrixType << ';'
            << report.n << ';'
            << (report.executed ? 1 : 0) << ';'
            << (report.executed ? ToString(report.status) : "пропущен") << ';'
            << report.rank << ';'
            << report.rowSwaps << ';'
            << report.elapsedMs << ';'
            << report.conditionNumberInf << ';'
            << report.conditionNote << ';'
            << report.solutionError.l1 << ';'
            << report.solutionError.l2 << ';'
            << report.solutionError.linf << ';'
            << report.residual.l1 << ';'
            << report.residual.l2 << ';'
            << report.residual.linf << ';'
            << report.note << '\n';
    }
}

}  // namespace

int main(int argc, char* argv[]) {
    try {
        SetupConsole();
        const bool runVeryLarge = HasFlag(argc, argv, "--run-10000");
        const std::uint64_t runSeed = GetRunSeed(argc, argv);
        std::mt19937_64 seedGenerator(runSeed);
        const auto nextExperimentSeed = [&seedGenerator]() {
            return seedGenerator();
        };

        std::vector<ExperimentReport> reports;

        std::cout << std::fixed << std::setprecision(6);
        std::cout << "Решение СЛАУ методом Гаусса\n";
           std::cout << "Стратегия выбора ведущего элемента: частичный выбор по
столбцу\n";
        std::cout << "Порог нуля: " << std::scientific
                  << kZeroTolerance << "\n";
        std::cout << "Базовый seed запуска: " << runSeed << "\n\n";
```

```
        {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomDiagonalDominantMatrix(
                3,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                3,
                DeriveSeed(seed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайный базовый пример 3x3",
                "случайная диагонально доминирующая",
                matrix,
                xExact,
                "Матрица и точное решение сгенерированы случайно.",
                seed
            ));
        }

        {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomPivotRequiredMatrix(
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                3,
                DeriveSeed(seed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайный пример 3x3 с обязательной перестановкой строк",
                "случайная плотная",
                matrix,
                xExact,
                  "Первый элемент в столбце занулен специально, поэтому требуется
перестановка строк.",
                seed
            ));
        }

        {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomRankDeficientMatrix(
                3,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                3,
                DeriveSeed(seed, 2)
            );
            const std::vector<double> b = Multiply(matrix, xExact);
            reports.push_back(RunExperiment(
                "Случайная вырожденная совместная система 3x3",
                "случайная вырожденная плотная",
                matrix,
                b,
                nullptr,
                  "Строки матрицы линейно зависимы, поэтому система совместна, но
решение не единственное.",
                seed
            ));
        }
```

```
        {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomRankDeficientMatrix(
                3,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                3,
                DeriveSeed(seed, 2)
            );
            std::vector<double> b = Multiply(matrix, xExact);
            b.back() += 1.0;
            reports.push_back(RunExperiment(
                "Случайная вырожденная несовместная система 3x3",
                "случайная вырожденная плотная",
                matrix,
                b,
                nullptr,
                            "Правая часть специально искажена, поэтому возникает
противоречивая строка.",
                seed
            ));
        }

        for (const std::size_t n : std::vector<std::size_t>{5, 8, 10}) {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomNearlySingularMatrix(
                n,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                n,
                DeriveSeed(seed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайная почти вырожденная матрица n=" + std::to_string(n),
                "случайная плохо обусловленная",
                matrix,
                xExact,
                    "Матрица построена как почти линейно зависимая, чтобы показать
влияние плохой обусловленности.",
                seed
            ));
        }

        for (const std::size_t n : std::vector<std::size_t>{5, 8, 20}) {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeHilbertMatrix(n);
            const std::vector<double> xExact(n, 1.0);

            reports.push_back(RunExperimentWithExactSolution(
                "Матрица Гильберта n=" + std::to_string(n),
                "матрица Гильберта",
                matrix,
                xExact,
                "Классический плохо обусловленный тест, точное решение выбрано как
x* = (1, ..., 1).",
                seed
            ));
        }

        {
            const std::uint64_t seed = nextExperimentSeed();
```

```
            std::size_t customN;
            std::cout << "Введите размер матрицы: ";
            std::cin >> customN;

            const DenseMatrix matrix = MakeRandomDenseMatrix(
                customN,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                customN,
                DeriveSeed(seed, 2)
            );

            reports.push_back(RunExperimentWithExactSolution(
                    "Пользовательская матрица " + std::to_string(customN) + "x" +
std::to_string(customN),
                "случайная плотная",
                matrix,
                xExact,
                "Размер матрицы введен пользователем.",
                seed
            ));
        }

        const std::uint64_t largeSeed = nextExperimentSeed();
        if (runVeryLarge) {
            const DenseMatrix matrix = MakeRandomDiagonalDominantMatrix(
                kVeryLargeN,
                DeriveSeed(largeSeed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                kVeryLargeN,
                DeriveSeed(largeSeed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайная диагонально доминирующая матрица 10000x10000",
                "случайная диагонально доминирующая",
                matrix,
                xExact,
                "Эксперимент был явно включен через флаг --run-10000.",
                largeSeed
            ));
        } else {
            reports.push_back(MakeSkippedLargeReport(largeSeed));
        }

        for (const auto& report : reports) {
            PrintReport(report);
        }

        WriteCsv("gauss_results.csv", reports);
        std::cout << "CSV-отчет сохранен в файл gauss_results.csv\n";
        return 0;
    } catch (const std::exception& error) {
        std::cerr << "Ошибка: " << error.what() << '\n';
        return 1;
    }
}
```

Файл methods.cpp

```
#include "methods.h"

#include <algorithm>
#include <cmath>
#include <limits>
#include <random>
#include <stdexcept>

namespace {

bool IsZeroRow(const DenseMatrix& matrix,
               std::size_t row,
               double zeroTolerance) {
    for (std::size_t col = 0; col < matrix.cols(); ++col) {
        if (std::abs(matrix(row, col)) > zeroTolerance) {
            return false;
        }
    }
    return true;
}

bool HasInconsistentRow(const DenseMatrix& matrix,
                        const std::vector<double>& b,
                        std::size_t startRow,
                        double zeroTolerance) {
    for (std::size_t row = startRow; row < matrix.rows(); ++row) {
        if (IsZeroRow(matrix, row, zeroTolerance) &&
            std::abs(b[row]) > zeroTolerance) {
            return true;
        }
    }
    return false;
}

}  // namespace

DenseMatrix::DenseMatrix() : rows_(0), cols_(0) {
}

DenseMatrix::DenseMatrix(std::size_t rows, std::size_t cols)
    : rows_(rows),
      cols_(cols),
```

```
      data_(rows * cols, 0.0) {
}

std::size_t DenseMatrix::rows() const {
    return rows_;
}

std::size_t DenseMatrix::cols() const {
    return cols_;
}

double& DenseMatrix::operator()(std::size_t row, std::size_t col) {
    return data_[row * cols_ + col];
}

double DenseMatrix::operator()(std::size_t row, std::size_t col) const {
    return data_[row * cols_ + col];
}

void DenseMatrix::swapRows(std::size_t first, std::size_t second) {
    if (first == second) {
        return;
    }
    for (std::size_t col = 0; col < cols_; ++col) {
        std::swap((*this)(first, col), (*this)(second, col));
    }
}

GaussianResult SolveGaussian(DenseMatrix A,
                             std::vector<double> b,
                             double zeroTolerance) {
    if (A.rows() != A.cols()) {
        throw std::invalid_argument("Matrix must be square.");
    }
    if (b.size() != A.rows()) {
        throw std::invalid_argument("Vector size must match matrix size.");
    }

    const std::size_t n = A.rows();
    std::vector<std::size_t> pivotColumns;
    std::size_t pivotRow = 0;
    std::size_t rowSwaps = 0;
```

```
for (std::size_t col = 0; col < n && pivotRow < n; ++col) {
    std::size_t bestRow = pivotRow;
    double bestValue = 0.0;

    for (std::size_t row = pivotRow; row < n; ++row) {
        const double candidate = std::abs(A(row, col));
        if (candidate > bestValue) {
            bestValue = candidate;
            bestRow = row;
        }
    }

    if (bestValue <= zeroTolerance) {
        if (HasInconsistentRow(A, b, pivotRow, zeroTolerance)) {
            GaussianResult result;
            result.status = SolutionStatus::NoSolution;
            result.rank = pivotColumns.size();
            result.rowSwaps = rowSwaps;
            return result;
        }
        continue;
    }

    if (bestRow != pivotRow) {
        A.swapRows(bestRow, pivotRow);
        std::swap(b[bestRow], b[pivotRow]);
        ++rowSwaps;
    }

    const double pivot = A(pivotRow, col);
    for (std::size_t row = pivotRow + 1; row < n; ++row) {
        const double factor = A(row, col) / pivot;
        if (std::abs(factor) <= zeroTolerance) {
            A(row, col) = 0.0;
            continue;
        }

        A(row, col) = 0.0;
        for (std::size_t inner = col + 1; inner < n; ++inner) {
            A(row, inner) -= factor * A(pivotRow, inner);
            if (std::abs(A(row, inner)) <= zeroTolerance) {
```

```
                A(row, inner) = 0.0;
            }
        }
        b[row] -= factor * b[pivotRow];
        if (std::abs(b[row]) <= zeroTolerance) {
            b[row] = 0.0;
        }

        if (IsZeroRow(A, row, zeroTolerance) &&
            std::abs(b[row]) > zeroTolerance) {
            GaussianResult result;
            result.status = SolutionStatus::NoSolution;
            result.rank = pivotColumns.size() + 1;
            result.rowSwaps = rowSwaps;
            return result;
        }
    }

    pivotColumns.push_back(col);
    ++pivotRow;
}

if (HasInconsistentRow(A, b, pivotRow, zeroTolerance)) {
    GaussianResult result;
    result.status = SolutionStatus::NoSolution;
    result.rank = pivotColumns.size();
    result.rowSwaps = rowSwaps;
    return result;
}

std::vector<double> x(n, 0.0);
for (std::size_t index = pivotColumns.size(); index > 0; --index) {
    const std::size_t row = index - 1;
    const std::size_t col = pivotColumns[row];

    long double rhs = b[row];
    for (std::size_t inner = col + 1; inner < n; ++inner) {
        rhs -= static_cast<long double>(A(row, inner)) * x[inner];
    }

    x[col] = static_cast<double>(rhs / A(row, col));
}
```

```
    GaussianResult result;
    result.status = (pivotColumns.size() == n)
        ? SolutionStatus::Unique
        : SolutionStatus::InfiniteSolutions;
    result.pivotStrategy = PivotStrategy::PartialByColumn;
    result.x = std::move(x);
    result.rank = pivotColumns.size();
    result.rowSwaps = rowSwaps;
    return result;
}

DenseMatrix MakeMatrix(std::size_t rows,
                       std::size_t cols,
                       const std::vector<double>& values) {
    if (rows * cols != values.size()) {
        throw std::invalid_argument("Row-major value count is incorrect.");
    }

    DenseMatrix matrix(rows, cols);
    for (std::size_t row = 0; row < rows; ++row) {
        for (std::size_t col = 0; col < cols; ++col) {
            matrix(row, col) = values[row * cols + col];
        }
    }
    return matrix;
}

DenseMatrix MakeHilbertMatrix(std::size_t n) {
    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
        for (std::size_t col = 0; col < n; ++col) {
            matrix(row, col) = 1.0 / static_cast<double>(row + col + 1);
        }
    }
    return matrix;
}

DenseMatrix MakeRandomDenseMatrix(std::size_t n,
                                  std::uint64_t seed,
                                  double minValue,
                                  double maxValue) {
```

```
    if (!(minValue < maxValue)) {
        throw std::invalid_argument("Random interval must be non-empty.");
    }

    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> distribution(minValue, maxValue);

    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
        for (std::size_t col = 0; col < n; ++col) {
            double value = distribution(rng);
            if (std::abs(value) < 0.1) {
                value = (value < 0.0) ? -0.1 : 0.1;
            }
            matrix(row, col) = value;
        }
    }
    return matrix;
}

DenseMatrix MakeRandomNearlySingularMatrix(std::size_t n,
                                           std::uint64_t seed,
                                           double perturbation) {
    if (perturbation <= 0.0) {
        throw std::invalid_argument("Perturbation must be positive.");
    }

    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> valueDistribution(-2.0, 2.0);
    std::uniform_real_distribution<double> scaleDistribution(0.5, 2.0);
    std::uniform_real_distribution<double> noiseDistribution(-1.0, 1.0);

    std::vector<double> baseRow(n, 0.0);
    for (double& value : baseRow) {
        value = valueDistribution(rng);
        if (std::abs(value) < 0.2) {
            value = (value < 0.0) ? -0.2 : 0.2;
        }
    }

    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
```

```
        const double scale = scaleDistribution(rng);
        for (std::size_t col = 0; col < n; ++col) {
            matrix(row, col) =
                scale * baseRow[col] + perturbation * noiseDistribution(rng);
        }
        matrix(row, row) += perturbation * static_cast<double>(row + 1);
    }
    return matrix;
}

DenseMatrix MakeRandomDiagonalDominantMatrix(std::size_t n,
                                             std::uint64_t seed,
                                             double minValue,
                                             double maxValue) {
    if (!(minValue < maxValue)) {
        throw std::invalid_argument("Random interval must be non-empty.");
    }

    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> distribution(minValue, maxValue);
    std::uniform_real_distribution<double> diagonalGap(1.0, 3.0);

    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
        double rowSum = 0.0;
        for (std::size_t col = 0; col < n; ++col) {
            if (row == col) {
                continue;
            }

            double value = distribution(rng);
            if (std::abs(value) < 0.05) {
                value = (value < 0.0) ? -0.05 : 0.05;
            }

            matrix(row, col) = value;
            rowSum += std::abs(value);
        }

        const double sign = (distribution(rng) >= 0.0) ? 1.0 : -1.0;
        matrix(row, row) = sign * (rowSum + diagonalGap(rng));
    }
```

```
    return matrix;
}

std::vector<double> Multiply(const DenseMatrix& matrix,
                             const std::vector<double>& x) {
    if (matrix.cols() != x.size()) {
        throw std::invalid_argument("Matrix/vector dimensions do not match.");
    }

    std::vector<double> result(matrix.rows(), 0.0);
    for (std::size_t row = 0; row < matrix.rows(); ++row) {
        long double sum = 0.0;
        for (std::size_t col = 0; col < matrix.cols(); ++col) {
            sum += static_cast<long double>(matrix(row, col)) * x[col];
        }
        result[row] = static_cast<double>(sum);
    }
    return result;
}

std::vector<double> Subtract(const std::vector<double>& left,
                             const std::vector<double>& right) {
    if (left.size() != right.size()) {
        throw std::invalid_argument("Vector dimensions do not match.");
    }

    std::vector<double> result(left.size(), 0.0);
    for (std::size_t index = 0; index < left.size(); ++index) {
        result[index] = left[index] - right[index];
    }
    return result;
}

std::vector<double> MakeRandomVector(std::size_t n,
                                     std::uint64_t seed,
                                     double minValue,
                                     double maxValue) {
    if (!(minValue < maxValue)) {
        throw std::invalid_argument("Random interval must be non-empty.");
    }

    std::mt19937_64 rng(seed);
```

```
    std::uniform_real_distribution<double> distribution(minValue, maxValue);

    std::vector<double> values(n, 0.0);
    for (double& value : values) {
        value = distribution(rng);
    }
    return values;
}

VectorNorms ComputeNorms(const std::vector<double>& values) {
    long double l1 = 0.0;
    long double l2Squared = 0.0;
    double linf = 0.0;

    for (double value : values) {
        const double absolute = std::abs(value);
        l1 += absolute;
        l2Squared += static_cast<long double>(absolute) * absolute;
        linf = std::max(linf, absolute);
    }

    VectorNorms norms{};
    norms.l1 = static_cast<double>(l1);
    norms.l2 = std::sqrt(static_cast<double>(l2Squared));
    norms.linf = linf;
    return norms;
}

namespace {

double ComputeMatrixInfinityNorm(const DenseMatrix& matrix) {
    double maxRowSum = 0.0;

    for (std::size_t row = 0; row < matrix.rows(); ++row) {
        long double rowSum = 0.0;
        for (std::size_t col = 0; col < matrix.cols(); ++col) {
            rowSum += std::abs(matrix(row, col));
        }
        maxRowSum = std::max(maxRowSum, static_cast<double>(rowSum));
    }

    return maxRowSum;
```

```
}

}  // namespace

bool ComputeConditionNumberInf(const DenseMatrix& matrix,
                               double zeroTolerance,
                               double& conditionNumber) {
    if (matrix.rows() != matrix.cols()) {
        throw std::invalid_argument("Matrix must be square.");
    }

    const std::size_t n = matrix.rows();
    const double normA = ComputeMatrixInfinityNorm(matrix);

    std::vector<long double> inverseRowSums(n, 0.0L);

    for (std::size_t col = 0; col < n; ++col) {
        std::vector<double> unitVector(n, 0.0);
        unitVector[col] = 1.0;

        const GaussianResult result =
            SolveGaussian(matrix, unitVector, zeroTolerance);

        if (result.status != SolutionStatus::Unique) {
            return false;
        }

        for (std::size_t row = 0; row < n; ++row) {
            inverseRowSums[row] += std::abs(result.x[row]);
        }
    }

    long double normInvA = 0.0L;
    for (long double rowSum : inverseRowSums) {
        normInvA = std::max(normInvA, rowSum);
    }

    conditionNumber =
        static_cast<double>(static_cast<long double>(normA) * normInvA);

    return std::isfinite(conditionNumber);
}
```

```
std::string ToString(PivotStrategy strategy) {
    switch (strategy) {
        case PivotStrategy::PartialByColumn:
            return "частичный выбор по столбцу";
    }
    return "неизвестная стратегия";
}

std::string ToString(SolutionStatus status) {
    switch (status) {
        case SolutionStatus::Unique:
            return "единственное решение";
        case SolutionStatus::InfiniteSolutions:
            return "бесконечно много решений";
        case SolutionStatus::NoSolution:
            return "решений нет";
    }
    return "неизвестный статус";
}

double EstimateDenseMatrixMiB(std::size_t n) {
    const long double bytes =
        static_cast<long double>(n) * n * sizeof(double);
    return static_cast<double>(bytes / (1024.0L * 1024.0L));
}

double EstimateAugmentedMatrixMiB(std::size_t n) {
    const long double bytes =
        static_cast<long double>(n) * (n + 1) * sizeof(double);
    return static_cast<double>(bytes / (1024.0L * 1024.0L));
}

long double EstimateGaussianFlops(std::size_t n) {
    const long double size = static_cast<long double>(n);
    return (2.0L / 3.0L) * size * size * size;
}
```

Файл methods.h

```
#pragma once
```

```
#include <cstddef>
#include <cstdint>
#include <string>
#include <vector>

class DenseMatrix {
public:
    DenseMatrix();
    DenseMatrix(std::size_t rows, std::size_t cols);

    std::size_t rows() const;
    std::size_t cols() const;

    double& operator()(std::size_t row, std::size_t col);
    double operator()(std::size_t row, std::size_t col) const;

    void swapRows(std::size_t first, std::size_t second);

private:
    std::size_t rows_;
    std::size_t cols_;
    std::vector<double> data_;
};

enum class PivotStrategy {
    PartialByColumn
};

enum class SolutionStatus {
    Unique,
    InfiniteSolutions,
    NoSolution
};

struct VectorNorms {
    double l1;
    double l2;
    double linf;
};

struct GaussianResult {
    SolutionStatus status = SolutionStatus::NoSolution;
```

```
    PivotStrategy pivotStrategy = PivotStrategy::PartialByColumn;
    std::vector<double> x;
    std::size_t rank = 0;
    std::size_t rowSwaps = 0;
};

GaussianResult SolveGaussian(DenseMatrix A,
                             std::vector<double> b,
                             double zeroTolerance = 1e-12);

DenseMatrix MakeMatrix(std::size_t rows,
                       std::size_t cols,
                       const std::vector<double>& values);

DenseMatrix MakeHilbertMatrix(std::size_t n);

DenseMatrix MakeRandomDenseMatrix(std::size_t n,
                                  std::uint64_t seed,
                                  double minValue = -10.0,
                                  double maxValue = 10.0);

DenseMatrix MakeRandomNearlySingularMatrix(std::size_t n,
                                           std::uint64_t seed,
                                           double perturbation = 1e-6);

DenseMatrix MakeRandomDiagonalDominantMatrix(std::size_t n,
                                             std::uint64_t seed,
                                             double minValue = -3.0,
                                             double maxValue = 3.0);

std::vector<double> Multiply(const DenseMatrix& matrix,
                             const std::vector<double>& x);
std::vector<double> Subtract(const std::vector<double>& left,
                             const std::vector<double>& right);

std::vector<double> MakeRandomVector(std::size_t n,
                                     std::uint64_t seed,
                                     double minValue = -5.0,
                                     double maxValue = 5.0);

VectorNorms ComputeNorms(const std::vector<double>& values);
```

```
bool ComputeConditionNumberInf(const DenseMatrix& matrix,
                               double zeroTolerance,
                               double& conditionNumber);

std::string ToString(PivotStrategy strategy);
std::string ToString(SolutionStatus status);

double EstimateDenseMatrixMiB(std::size_t n);
double EstimateAugmentedMatrixMiB(std::size_t n);
long double EstimateGaussianFlops(std::size_t n);
```